Technical report · Version 1.0 · October 2026

ConveySort-Arm: A Low-Cost Vision-Guided Sorting Cell with a Conveyor Belt and a Six-Servo Arm

Design and Prototype

Jewel Nath[1,2], **Shafiul Alam Chowdhury**[2]

[1]Independent researcher, Dhaka, Bangladesh [2]RoboAI, Bangladesh

Jewel Nath: ORCID 0009-0002-6609-6378 · Code: https://github.com/jewel-nath/conveysort-arm

Licence: CC BY 4.0

Abstract

ConveySort-Arm is a small sorting cell built in 2024 at RoboAI, Bangladesh, from low-cost parts: a conveyor belt turned by a stepper motor, an ultrasonic sensor at the belt's end, a webcam above it, and a six-servo arm beside it. It brings each block to the arm, measures its size from above, and has the arm set it down in the place for that size.

The work is split between a PC and an Arduino Uno. On the PC, a Python program with OpenCV thresholds each camera frame, finds the block's outline, and calls it small, medium or large by its area; once ten more frames agree on the same size, it sends the size to the Arduino over serial. The Arduino runs the belt until the block reaches the sensor, then moves the arm through a fixed 15-step pick-and-place, with a grip and a set-down place for each size.

The cell was built and run in October and November 2024; videos show the belt bringing blocks to the arm, OpenCV sizing them, and the arm picking them up and setting them down. As programmed, one sort takes 35 to 38 seconds. This report describes the build, the electronics, the vision, the control and the timing of the cell, and sets out the lessons and the next steps, the first being to run the arm from the ROS 2 workspace written for it.

Keywords: robotic sorting, pick and place, conveyor, computer vision, OpenCV, Arduino, servo arm, low-cost automation.

1 Introduction

Sorting objects on a conveyor is one of the oldest jobs for robots in industry: a camera or a sensor tells the objects apart, and an arm moves each one to its place. Research on picking has moved on to unknown objects in clutter [1, 2], but the basic loop, sense, decide, pick and place, is still the core of the task, and still the best way to learn it.

ConveySort-Arm is a small, low-cost version of that loop (Fig. 1). A belt brings each block to an arm; a webcam and OpenCV measure the block; and the arm sets it down in one of three places by its size. It was built at RoboAI from parts available in Bangladesh: hobby servos, a stepper motor, an ultrasonic

sensor, an Arduino and a webcam.

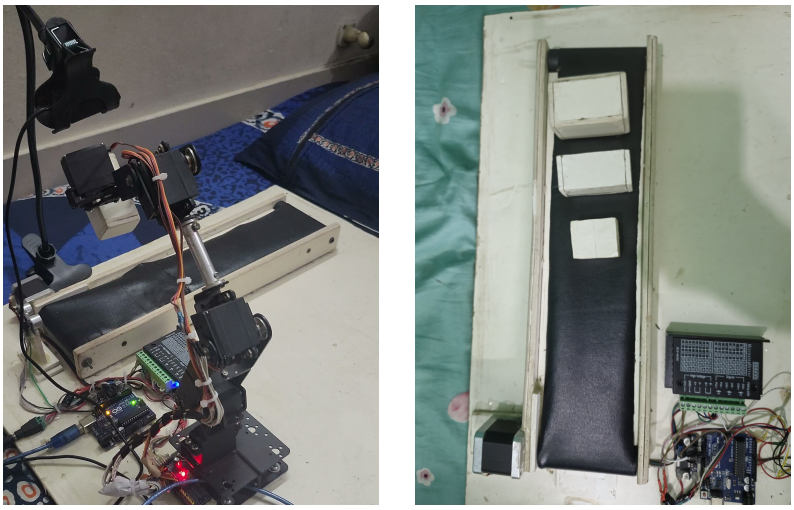


Figure 1: ConveySort-Arm. The arm lifting a block off the belt, the webcam on its clamp above, on 18 November 2024 (left). The conveyor from above with the three test sizes, the stepper motor at the near end, and the driver and the Arduino beside it, on 16 November 2024 (right).

This report describes the cell as built, and contributes:

1. a sorting cell built from low-cost parts: a stepper-driven belt, an ultrasonic stop, a webcam and a six-servo arm (Sections 4–5);
2. size sorting by outline area with OpenCV, steadied over eleven frames in a row before a decision (Section 6);
3. a simple split of the work between a PC, for vision, and an Arduino, for the belt and the arm, joined by a one-character serial protocol (Section 7);
4. the timing of a sort, computed from the firmware (Section 8);
5. the behaviours shown on video, and the open source code, under the MIT licence, with its course-derived ROS 2 workspace under Apache 2.0 [3] (Section 10).

Results at a glance

What the cell does	How	Evidence
Brings each block to the arm	the belt runs while the sensor sees a block, and stops it at 7 cm	shown on video
Sizes each block from above	OpenCV outlines the block and sizes it by its area	shown on video
Picks the block up and sets it down	a 15-step sequence on six servos	shown on video
Puts each size in its own place	a grip and a base angle for each size: 90°, 50° or 10°	the team's runs
Sorts one block in 35–38 s	as programmed, from the command to home	computed from the code

The videos were recorded on 11, 16 and 18 November 2024. "The team's runs" are the authors' account of their November 2024 tests.

2 Related Work

Picking with vision. Saxena et al. learned grasp points for objects a robot had never seen from images [1], and Zeng et al. picked and placed unknown objects from clutter with learned grasp affordances [2]. ConveySort-Arm works in the simpler setting those methods grew from: known objects, one at a time, in a fixed place on a belt, so that a fixed motion can pick each one.

Measuring objects in images. OpenCV [4] provides the image operations the cell uses: conversion to grey, blurring, thresholding and outline finding, the last by Suzuki and Abe's border following [5]. The block's size is the area inside its outline.

Arms on ROS. ROS 2 [6] and MoveIt [7] are the usual way to model and plan an arm today. The cell's ROS 2 workspace follows Antonio Brandi's course on manipulators [8] (Section 9).

3 System Overview

Fig. 2 shows the cell. A PC reads the webcam and runs the vision program; it sends the Arduino one character, the block's size, over USB serial. The Arduino runs everything that moves: it drives the belt's stepper motor through a TB6600 driver, reads the ultrasonic sensor at the belt's end, and moves the arm's six servos through a PCA9685 servo driver [9]. It prints a line for each event, which the PC reads back.

The split keeps each side simple. The PC needs no real-time control, and the Arduino needs no camera: the two agree only on four characters, and the Arduino acts on a size only when the belt has stopped with a block in place.

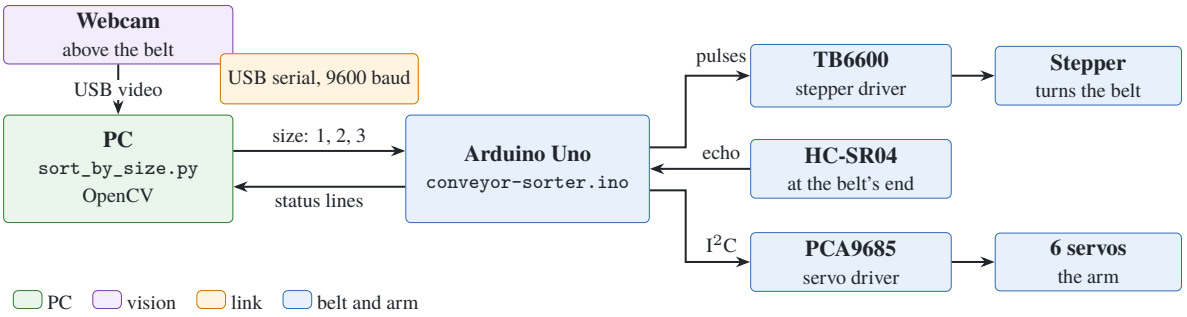


Figure 2: System overview. The PC (green) runs the vision (purple) and sends the block's size over the serial link (orange); the Arduino drives the belt and the arm (blue).

4 Mechanical Build

The cell grew in two months (Figs. 3 and 4). The first version, in early October 2024, put a cloth belt, its stepper motor, the sensor, the Arduino and the arm on one board. By November the belt was a black band in a wooden frame, ending at the ultrasonic sensor, with the arm beside its end. The webcam came last: on a stand above the belt on 16 November, then on a clamp by 18 November.

The arm has six servos: a base, a shoulder, an elbow, a wrist that pitches and rolls, and a gripper. It stands beside the end of the belt, so it can reach the stopped block and three set-down places around its base.

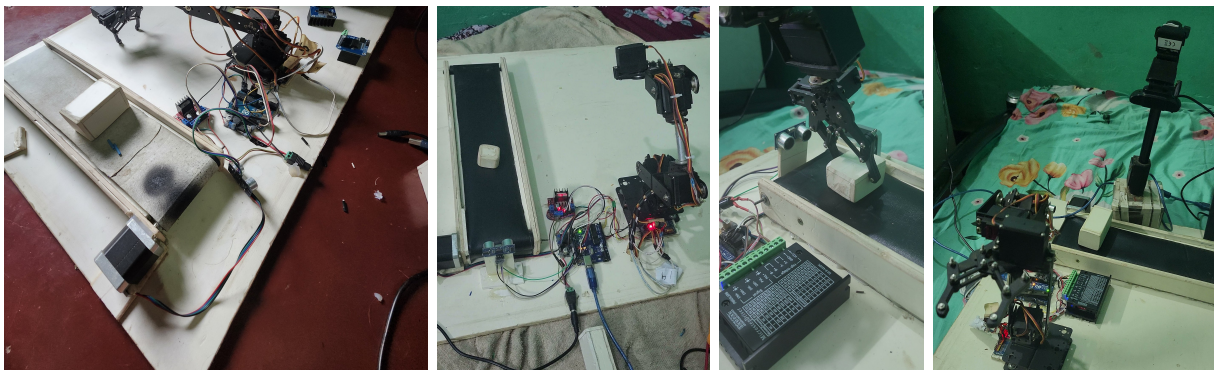


Figure 3: The build, in order: the first belt, the sensor, the Arduino and the arm on one board (5 October); the black belt bringing a block to the sensor (11 November); the gripper closing on a block under the sensor, the TB6600 driver in front (16 November); the webcam on its first stand above the belt (16 November).

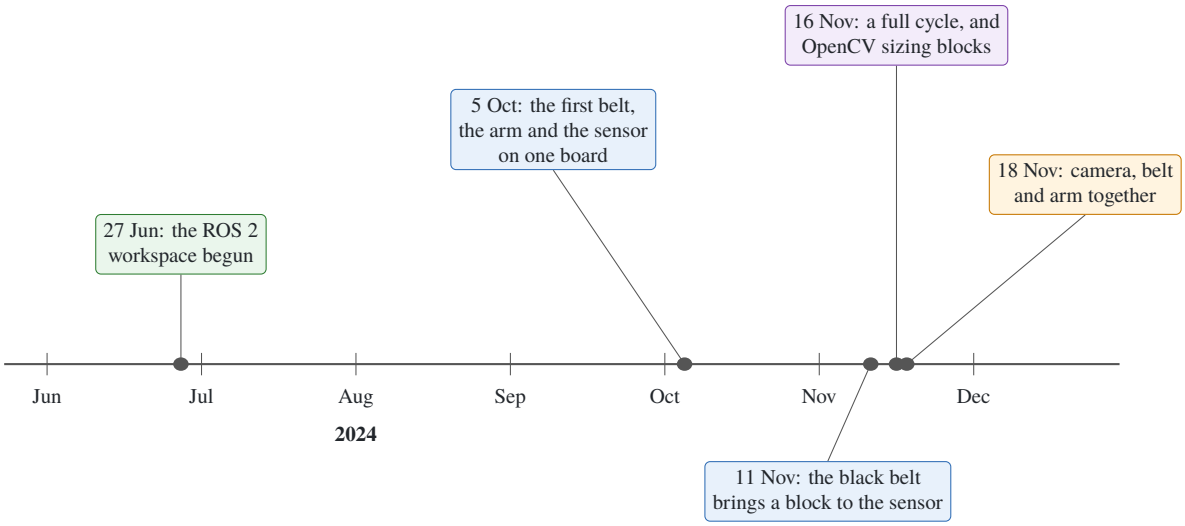


Figure 4: ConveySort-Arm in 2024, from the repository's history and the dated photos and videos.

5 Electronics

Fig. 5 gives every connection, from the firmware. The stepper driver takes a STEP pulse on pin 5 and a direction on pin 6; the ultrasonic sensor is triggered on pin 9 and answers on pin 8. The PCA9685 sits on the Arduino's I^2C pins and drives the six servos at 60 Hz, on its channels 8, 2, 3, 5, 6 and 7, from the base to the gripper. A servo's angle from 0 to 180° maps to 150–650 of the driver's 4096 counts.

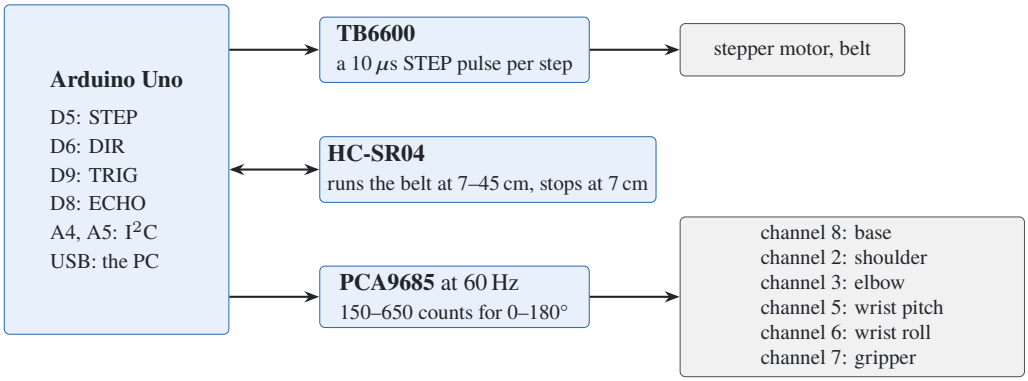


Figure 5: The Arduino's connections, from `firmware/conveyor-sorter`: the belt's driver, the sensor, and the servo driver with each joint's channel.

6 Vision: Sizing a Block

From a frame to an outline. The PC reads the webcam at 639 by 480 pixels and mirrors each frame (Fig. 6). It turns the frame grey and keeps a window that leaves out the edges: 100 pixels at the top, 80 at the bottom and 20 at each side, to leave out the image's edges, such as the belt's frame. A 9 by 9 Gaussian blur smooths it, and a threshold at 230 of 255 keeps only the white block on the black belt. OpenCV then finds the outer outlines [5].

Filtering. An outline counts as a block only if its area is at least 1,200 square pixels, both sides of its bounding box are at least 20 pixels, its proportions lie between 1:4 and 4:1, and it lies inside the window. Of the outlines left, the largest is the block.

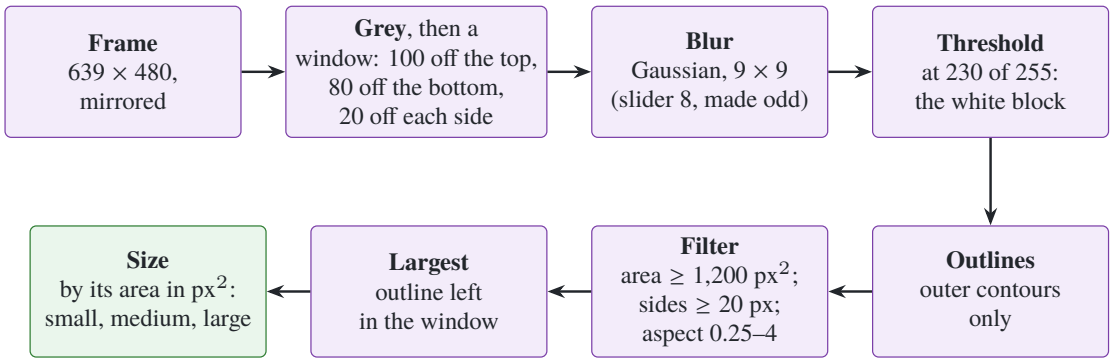


Figure 6: How the PC sizes a block, from `src/vision/sort_by_size.py`. The threshold and the blur were tuned live with sliders.

Size by area. The block's size is the area inside its outline (Fig. 7): under 17,000 square pixels it is small, under 28,000 medium, and anything larger is large; that is, under 5.5% and 9.1% of the frame. The limits suit the three test blocks with the camera at its working height; they are areas in the image, not sizes in millimetres.

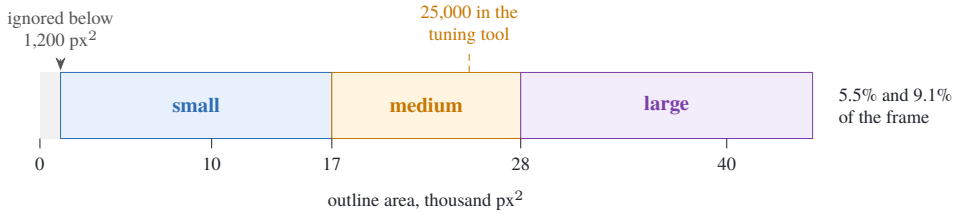


Figure 7: The size classes by outline area. The tuning tool, `object_size_detection.py`, uses 25,000 square pixels for the medium limit; the sorter uses 28,000.

Deciding. A single frame can be wrong, while a hand passes or the block is still moving. The program therefore counts how many frames in a row give the same size, and sends a size only when ten more frames agree with the first, eleven in a row, and at most once every five seconds (Fig. 8).

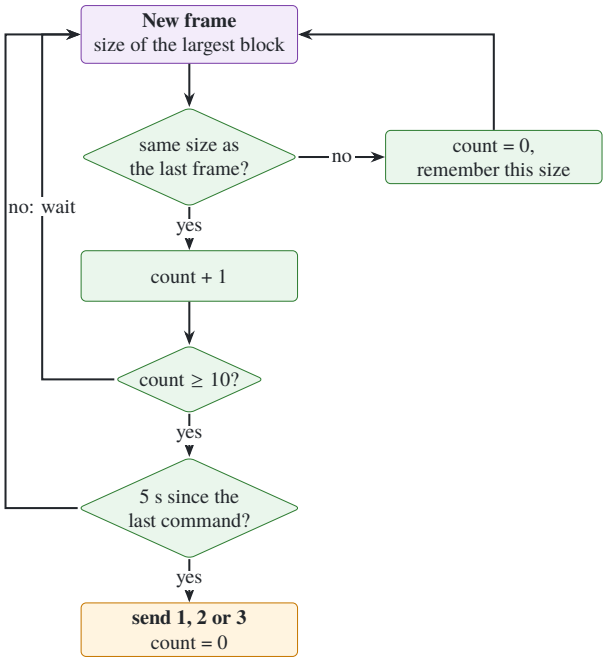


Figure 8: When the PC sends a size: after ten steady frames, and no more than once every five seconds.

7 Control

The belt. The Arduino reads the ultrasonic sensor every 100 ms, as the mean of three pings (Fig. 9). With a block 7 to 45 cm away, it runs the belt; when the block is within 7 cm, at the belt's end, it stops the belt and reports "Object detected in position". Once the block is taken away, the belt waits for the next one. A belt that runs for 30 seconds without a block arriving is reset, and the arm is sent home.

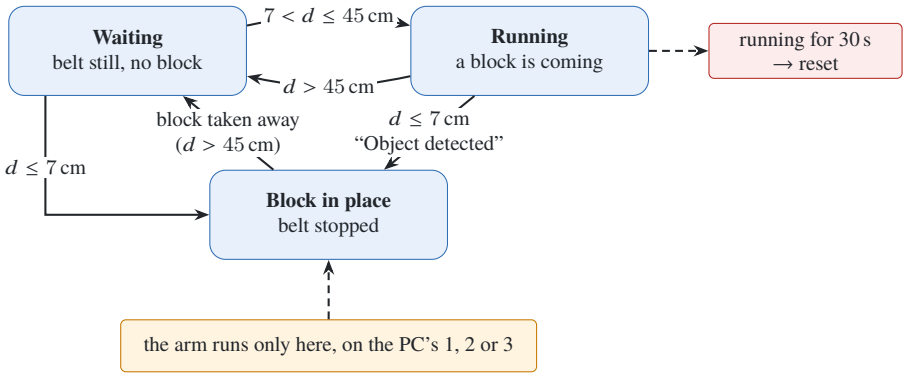


Figure 9: The belt's states, from the distance d the ultrasonic sensor reads.

The serial protocol. The PC and the Arduino talk in single characters at 9600 baud: `0` sends the arm home, and `1`, `2` and `3` ask it to sort a small, a medium or a large block (Fig. 10). The Arduino ignores a size, with "No object detected", unless the belt has stopped with a block in place, so a size sent early does no harm.

The pick and place. The arm sorts a block in 15 steps, each moving one joint (Fig. 11). It reaches over the belt and down, grips the block, lifts it, turns its base to the block's place, lowers it, and lets go; then it

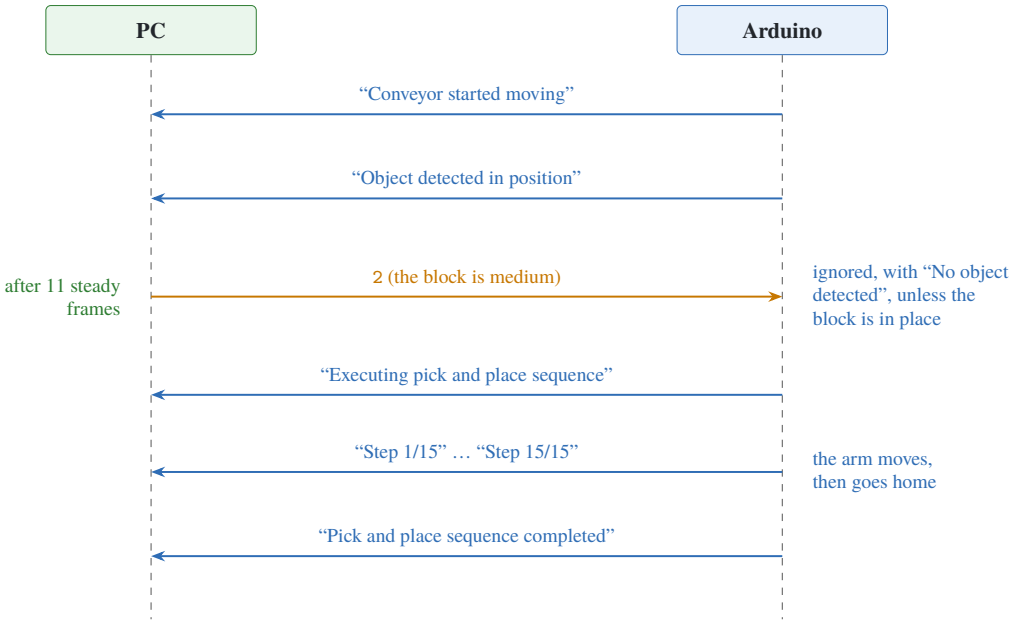


Figure 10: One sort, as the messages over the serial link: the Arduino’s lines (blue) and the PC’s size (orange).

lifts and returns home, joint by joint. Two steps depend on the size: the grip closes to 155°, 150° or 140°, wider for a larger block, and the base turns to 90°, 50° or 10° to set it down in its own place.

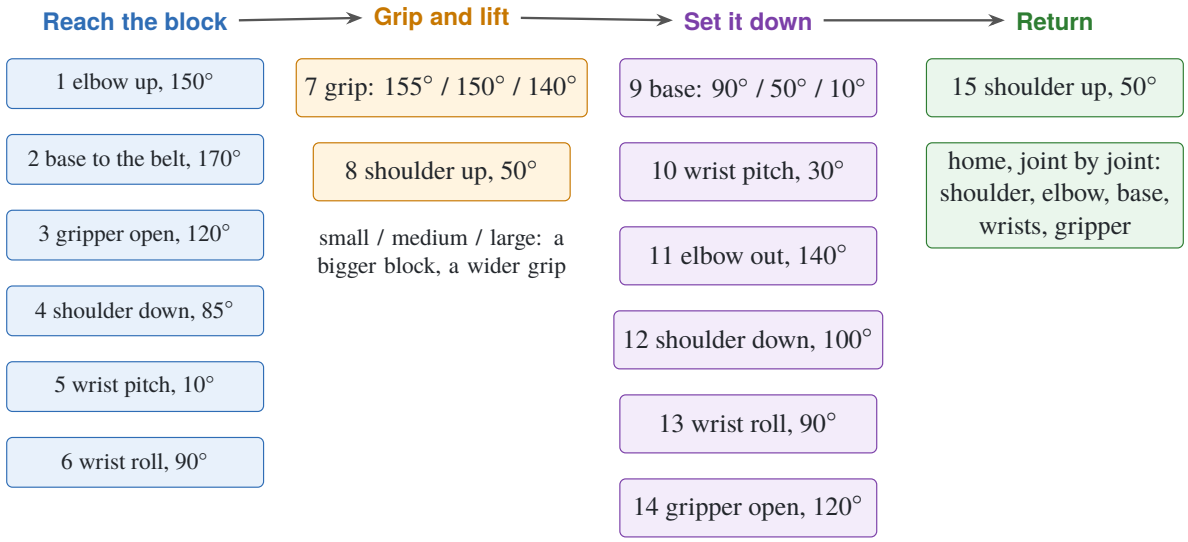


Figure 11: The 15-step pick-and-place, from the firmware. Steps 7 and 9 depend on the block’s size.

Smooth, safe motion. Each joint moves in steps of 5 counts, about 1.8°, every 40 ms, about 45° per second, and the firmware pauses for a second after each step. After a move, that joint’s angle is saved to the Arduino’s EEPROM if five seconds have passed since the last save, and at start-up the firmware reads the saved angles back and moves the arm home from them. A watchdog and an error handler send the arm home after a sensor error, a belt time-out or an invalid move.

8 Timing

Because every move and pause is fixed in the firmware, the time of a sort can be worked out exactly. Replaying the sequence gives the commanded angle of each joint over time (Fig. 12) and the length of each phase (Fig. 13). One sort, from the command to the arm back home, takes 35.0 s for a small block, 36.5 s for a medium one and 37.9 s for a large one, leaving out the few tenths of a second the bus writes take; the larger blocks go to places further round, so the base turns further. More than half of it, 21 s, is the one-second pause after each of the 21 moves.

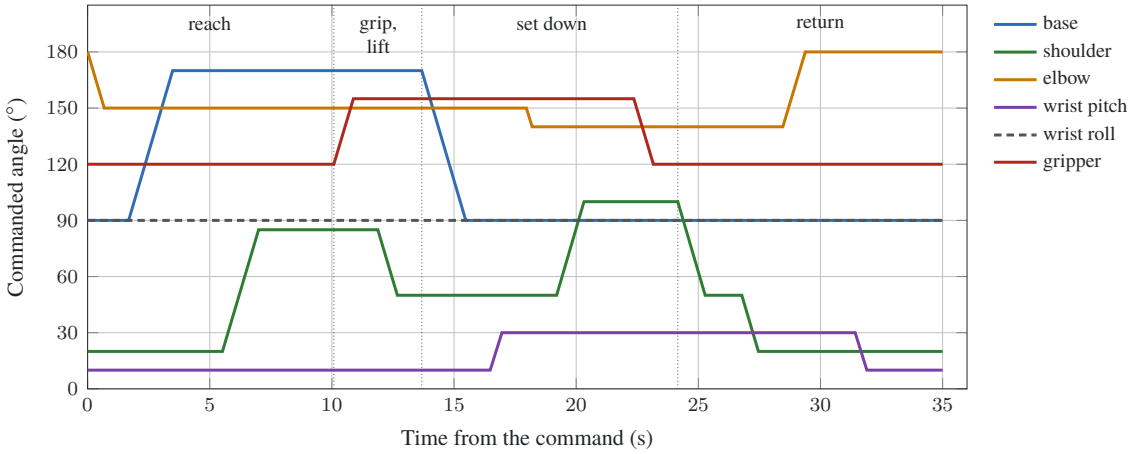


Figure 12: Commanded joint angles over one sort of a small block, computed from the firmware. The dotted lines mark the phases.

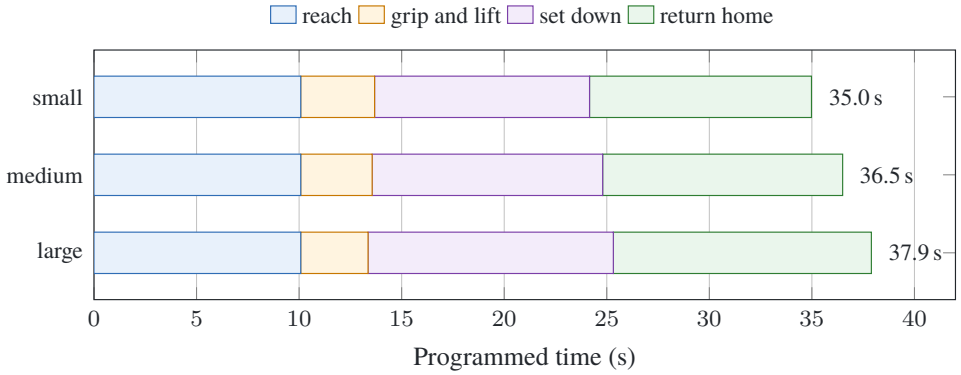


Figure 13: The programmed time of one sort, by block size, in its four phases, computed from the firmware.

The videos are close to this. In the original 57 s video of 16 November, read by eye, the gripper is closed on the block by 28 s and has set it down by 42 s, about 14 s later; the firmware gives 13.1 s from the start of the grip to the end of the release for a small block. The replay is of the code as committed to the repository, in April 2026.

9 The ROS 2 Workspace

Alongside the sorter, the repository holds a ROS 2 Humble workspace for the arm (Appendix, Fig. 16): its model in URDF, a ros2_control hardware interface, a MoveIt set-up, launch files for RViz, Gazebo and the real arm, and a task server. It is adapted from Antonio Brandi's course [8], with the hardware interface, the controllers, the slider node and the arm's sketch changed for six servos in place of four, and the arm's model taken from EnviroBot [10]. Running the arm from it was the team's next step; the project ended first, with the sorter working on its own, so the workspace was written but not run. As written, its task presets still fit the course's arm, and its serial link to the Arduino needs fixing before it can drive this arm.

10 Demonstrations

The cell's work was recorded on video in November 2024 (Table 1, Fig. 14).

Table 1: Behaviours recorded on video.

Date	What the video shows
11 Nov 2024	One belt cycle: a block rides the belt to the sensor and stops (original, 51 s)
16 Nov 2024	One full cycle before the camera: the belt brings a block, the arm grips it and sets it down (original, 57 s)
16 Nov 2024	The tuning window: three blocks placed on the belt in turn by hand, each outlined, measured and called large, medium or small (14 s)
18 Nov 2024	The whole cell: the webcam above the belt, the arm taking a block off it, setting it down and going home (original, 178 s)

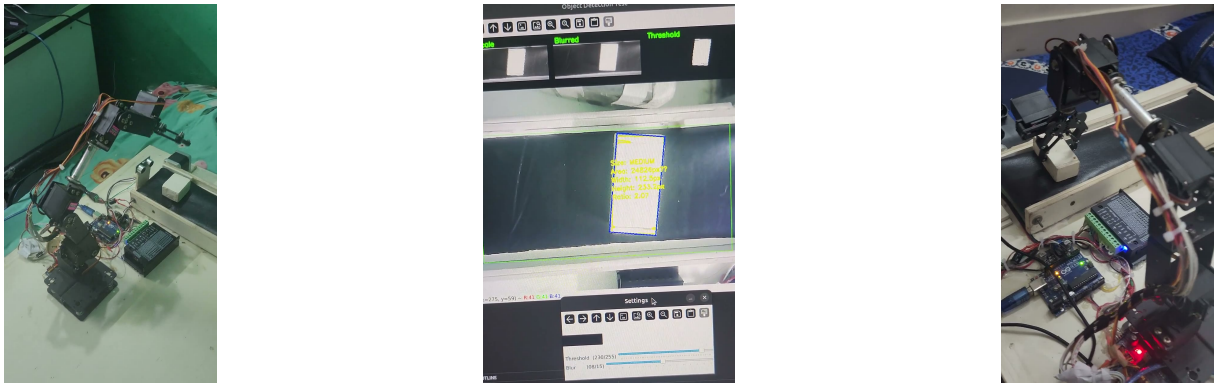


Figure 14: From the videos: the arm at the end of the belt (16 November); the tuning window outlining and sizing a block (16 November); the whole cell at work (18 November).

11 Lessons Learned

A physical stop makes picking easy. Stopping each block at the same point, with an ultrasonic sensor, let a fixed sequence pick every block. No grasp planning was needed.

Let each board do what it does well. The PC handled the camera and OpenCV; the Arduino handled timing and motors. One character per sort was all they needed to share.

Steady the decision, not only the image. Waiting for eleven agreeing frames, and acting only when the block is in place, is meant to keep a passing hand or a moving block from triggering a sort, with very little code; the tuning video shows how readily a hand is outlined.

Slow is reliable, but pauses add up. Small steps and a pause after each were chosen to keep the hobby servos steady, at the cost of a sort that takes over half a minute.

12 Limitations and Future Work

ConveySort-Arm was built to work, and its runs were recorded on video rather than counted: its sorting accuracy and its cycle time over many blocks are not yet measured, and its timing here is computed from the firmware. Its sizes are areas in the image, set for one camera height, and its motion is a fixed sequence.

The open steps follow from these (Fig. 15): running the arm from the ROS 2 workspace, measuring accuracy and cycle time over many blocks, and calibrating the camera so that sizes are in millimetres; then planning the motion with MoveIt, grasping where the camera sees the block, and sorting by colour and shape as well as size.

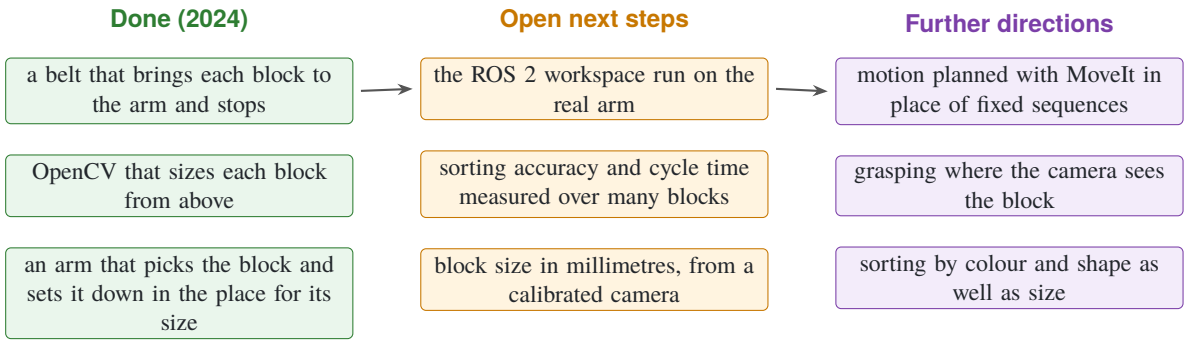


Figure 15: What ConveySort-Arm showed in 2024, and the open steps.

13 Conclusion

ConveySort-Arm shows that a working sorting cell can be built from hobby parts and about a thousand lines of code: a belt that stops each block at the arm, a webcam and OpenCV that size it, and a six-servo arm that sets it down in the place for its size. Its split between a PC for vision and an Arduino for motion, joined by one character per sort, keeps each part simple, and its code is open for others to build on.

Author Contributions

Jewel Nath and Shafiul Alam Chowdhury designed, built and tested ConveySort-Arm together at RoboAI. Jewel Nath put its code into its public repository and wrote this report.

Acknowledgements

The ROS 2 workspace, three course sketches and the sketch for running the real arm from ROS 2 are adapted from Antonio Brandi's course "Robotics and ROS 2 – Learn by Doing! Manipulators" (Apache 2.0), and each derived file keeps its notice. The ESP32 test sketch comes from Ujwal Nandanwar's RobotArm (MIT), and the development container from the ROS 2 documentation's VS Code guide (CC BY 4.0). The arm's model comes from the first author's earlier EnviroBot project [10], built at Comilla University with Tasmiara Jahan Toma and Tanjina Akter, where no licence was stated for it; its meshes predate it, and their first source is not known.

This report was written with the help of AI assistants, under the first author's direction; its statements were checked against the code, its history and the dated photos and videos.

References

[1] A. Saxena, J. Driemeyer, and A. Y. Ng, "Robotic grasping of novel objects using vision," *The International Journal of Robotics Research*, vol. 27, no. 2, pp. 157–173, 2008.

[2] A. Zeng, S. Song, K.-T. Yu, E. Donlon, F. R. Hogan, M. Bauza *et al.*, "Robotic pick-and-place of novel objects in clutter with multi-affordance grasping and cross-domain image matching," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 3750–3757.

[3] J. Nath, "ConveySort-Arm: Source code," https://github.com/jewel-nath/conveysort-arm, 2026, MIT licence; course-derived files Apache-2.0.

[4] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[5] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985.

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[7] D. Coleman, I. A. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: a MoveIt! case study," *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.

[8] A. Brandi, "Robotics and ROS 2 – learn by doing! manipulators," https://github.com/AntoBrandi/Robotics-and-ROS-2-Learn-by-Doing-Manipulators, 2023, course code, Apache-2.0 licence.

[9] NXP Semiconductors, "PCA9685: 16-channel, 12-bit PWM Fm+ I^2C-bus LED controller," Product data sheet, Rev. 4, 2015.

[10] J. Nath, T. J. Toma, and T. Akter, "EnviroBot: A mobile robotic arm that detects and picks up waste – design, YOLOv8 waste detection and prototype," Technical report, Zenodo, 2026.

A Additional Material

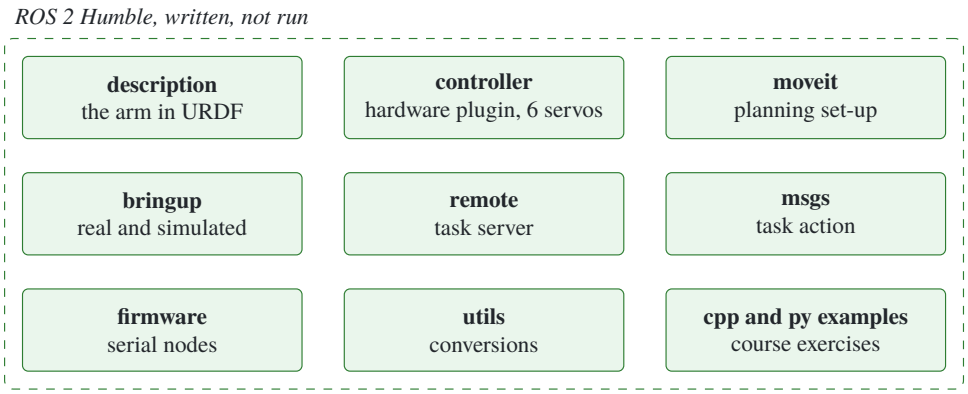


Figure 16: The ROS 2 Humble workspace, adapted from Antonio Brandi's course: ten packages, written but not run.

Table 2: The repository.

Folder	Holds
`src/vision/`	the sorter (`sort_by_size.py`) and the tuning tool
`firmware/conveyor-sorter/`	the Arduino program for the belt and the arm
`src/conveysort_*/`	the ROS 2 workspace (course-derived, Apache 2.0)
`experiments/`	the first belt, sensor and servo tests
`docs/media/`	the photos and videos
`docs/report/`	this report, its figures and how to build it
`docs/diagrams/`	every diagram as its own PDF and PNG